



Savitribai Phule Pune University
Centre For Distance and Online Education

SAVITRIBAI PHULE PUNE UNIVERSITY, PUNE
CENTRE FOR DISTANCE AND ONLINE EDUCATION

Program	Master of Computer Application	
Course	MJ Programming from First Principles	
Topic Name	The types of expressions may be determined by simple examination of the program text, understanding such rules (practice session), Part 2	
Topic Code	-	
Unit/Module No. & Name	9	Inference
Self-Learning Hours	-	

Topic Details

S.No	Topic	Pg.No
1.0	Introduction	4-5
2.0	Meaning And Definition	6-8
3.0	Nature Or Type	9-12
4.0	Examples And Case Studies	13-14
5.0	Keywords And Touch Vocabulary	15



OBJECTIVE

1. Program text is the written form of instructions that tell a computer what actions to perform.
2. It is made of symbols that follow clear and agreed rules.
3. These rules are not only about what is right or wrong.
4. They also help people read code, check it carefully, and improve it over time.
5. In this unit, you will learn how to describe program text in an academic way using simple language and clear examples.
6. The objective of this unit is to build a strong foundation for courses in programming, compilers, software engineering, and data-related work.
7. When you understand why rules exist, you can write code that is correct and easy to maintain.
8. You will learn that program rules can be grouped into different types.
9. These include rules for spelling tokens, rules for structure, and rules for meaning.
10. By the end of this lesson, you should be able to complete several tasks.
11. You should be able to explain what program text is and how it is different from normal writing.
12. You should be able to define a programming language rule and classify it as lexical, syntax, semantic, or style related.
13. You should be able to read a short code sample and identify the part that breaks a rule.
14. You should be able to describe how tools such as a compiler, interpreter, and linter check rules.
15. You should also be able to write a short paragraph explaining why rules help both computers and humans.

1.0 INTRODUCTION

A computer does not understand ideas the way humans do. It works by following symbols and steps exactly. A program is the way we give these steps to the computer. The program is not only an idea in the programmer's mind. It must be written in a form that the machine can read and process. This written form is called program text. A program may be very small or extremely large. In all cases, the computer requires the program text to follow clear rules.

In academic study, program text is treated as a formal object. This means it can be described in a clear and exact way. We can explain how it is written, how it should appear, and what it means. This is useful because it gives the same result every time. If two students write the same program text, the computer will behave in the same way, as long as the environment is the same.

1.1 Why Program Rules Matter

Rules are important because they give order and structure. Without rules, the computer would not know where one instruction ends and another begins. Rules also allow tools to find errors early. A compiler can report a missing symbol. An interpreter can stop when a name is not defined. A linter can warn about unsafe or poor coding patterns. In each case, rules help prevent small mistakes from becoming serious problems.

Rules also help when people work together. In group projects, many people read and change the same program text. If everyone writes in a different way, the code becomes hard to understand. Shared rules, such as naming rules or indentation rules, make the program text consistent. Consistent code is easier to read, test, and fix.

1.1.1 Humans Vs Machines

Humans can often understand meaning even when a sentence has small mistakes. For example, a person can still understand a message with a missing comma. A computer cannot do this. If a rule requires a semicolon, then leaving it out can stop the program from running. This strict behavior may seem difficult, but it is useful. It makes program behavior predictable. Predictable behavior is important for safe and correct software.

1.2 Program Text as Communication

Program text is written for two readers. One reader is the computer. The other reader is a human. A program must run correctly on the machine, but it should also be easy for people to read. In universities, readable code helps learning because others can follow the logic. In industry, readable code helps maintenance because another person may work on the same code later.

One helpful idea is to think of code as a technical document. A technical document uses clear terms, explains symbols, and follows a clear structure. Program text should do the same. It should use clear names, clear blocks, and simple control flow. It should also avoid surprises, such as hidden actions that change data in unexpected ways.

1.2.1 Reading Code Like a Document

When reading a research paper, you expect clear definitions, logical order, and supported ideas. The same approach can be used when reading program text. You can look for definitions such as variables and function headers. You can look for structure such as loops and conditions. You can look for support such as tests and examples. In this way, learning program rules is not only a technical skill. It is also a reading and writing skill used in academic study.



2.0 MEANING AND DEFINITION

2.1 Meaning of Program Text

Program text, often called source code, is the human readable form of a program. It is stored in files, shared in repositories, and processed by tools. It includes symbols that represent operations, data, and control. It also includes spaces and line breaks that help separate parts of the code. In many languages, comments are also part of the program text. Comments do not change execution, but they can change how well the code is understood.

Program text is not random writing. It is constrained by the rules of a programming language. Each language has its own rules, but many ideas are shared. For example, most languages have names for variables, operators for arithmetic, and keywords for control flow. The key point is that the text must follow patterns that tools can process.

2.1.1 Tokens and Lexemes

To analyze program text, tools often begin by breaking the text into tokens. A token is a category such as IDENTIFIER, NUMBER, KEYWORD, or OPERATOR. The exact characters that make up one token instance are sometimes called a lexeme. For example, the token category IDENTIFIER could have the lexeme `totalCost`. The token category NUMBER could have the lexeme `123`.

This first step is called lexical analysis. It matters because later steps depend on it. If the lexer cannot form valid tokens, the rest of the tool chain cannot continue. For students, tokens are also a useful idea because they help you see code as structured text, not as a long string of characters.

2.2 Definition of Rules

A programming language rule is a precise constraint on how program text can be formed and what it means. Rules can be written in a reference manual, taught in a course, or encoded in a tool. Some rules are strict and required for execution. Other rules are conventions chosen for readability and safety. In academic writing, it is useful to state what kind of rule you mean, because that clarifies whether the rule is enforced by the language or by a style policy.

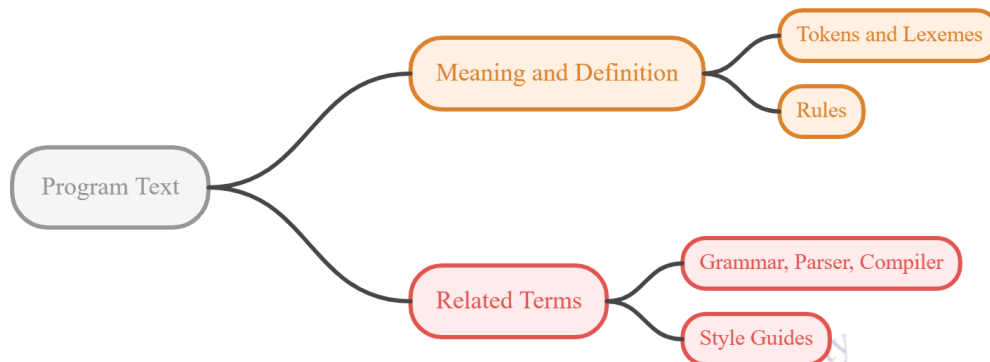
2.2.1 Syntax Rules

Syntax rules describe the structure of program text. They define how tokens can be arranged to form statements and expressions. For example, a language may require that a function call looks like name followed by parentheses and arguments. A language may require that an if statement

includes a condition and a block. A language may require braces around blocks, or it may require indentation.

Fig. 1

Program Text and Rules



The diagram shows program text connected to meaning, rules, and related terms. It highlights tokens and lexemes, grammar rules, parsers, compilers, and style guides, explaining how written code follows rules to create correct, readable, and well-structured programs.

Syntax is like the grammar of a language, but it is more strict. Small differences in punctuation can change the structure or make the text invalid. In many courses, students first meet rules through syntax errors, such as a missing parenthesis or an unmatched brace. Learning syntax rules reduces these errors and helps you write code more confidently.

2.2.2 Semantic Rules

Semantic rules describe meaning. They define what the code does when it runs, and they define whether an operation is allowed in a meaningful way. For example, a semantic rule may say that you cannot use a variable before it is defined. Another rule may say that you cannot add a number to a function. A semantic rule may also define how expressions are evaluated, such as left to right evaluation or short circuit behavior in boolean operations.

Semantics matters because two pieces of code can have the same syntax shape but different meaning, depending on names, types, and context. When students debug programs, many bugs

are semantic bugs. The code may run, but it may do the wrong thing. Understanding semantic rules helps students explain why a program behaves as it does.

2.3 Related Terms

2.3.1 Grammar, Parser, Compiler

A grammar is a set of formal rules that describe valid structures in a language. A parser is a tool or algorithm that checks a token stream against the grammar and builds a structure called a parse tree or syntax tree. A compiler is a system that translates program text into another form, often machine code or an intermediate code. An interpreter executes program text directly, often after building internal representations.

These tools enforce rules. The lexer enforces lexical rules. The parser enforces syntax rules. The semantic checker enforces semantic rules. Then later phases can optimize code or generate target code. Understanding this pipeline helps students understand error messages. A “syntax error” often comes from parsing, while a “type error” often comes from semantic checking.

2.3.2 Style Guides

A style guide is a collection of conventions for writing program text in a consistent way. It can include indentation width, naming patterns, line length, and comment style. A style guide is not always required by the language, but it is often required by a course or a workplace. Style rules reduce friction in code review and help tools such as formatters produce stable output.

3.0 NATURE OR TYPE

3.1 Lexical Rules

Lexical rules define the smallest units of program text. They define how identifiers are written, how numbers are written, and how comments are marked. For example, a lexical rule may say that an identifier starts with a letter or underscore, followed by letters, digits, or underscores. Another lexical rule may say that an integer is a sequence of digits.

Lexical rules also define how to treat whitespace. Some languages treat whitespace as mostly optional, while others use it to mark blocks. Lexical rules can also define escape sequences in strings, like `\n` for a new line character.

3.1.1 Whitespace and Indentation

Whitespace is space, tab, and new line characters. In many languages, whitespace separates tokens but does not carry meaning beyond separation. In some languages, whitespace can be significant. For example, in Python, indentation is part of the block structure. If a line is indented more, it belongs to a deeper block. If indentation is wrong, the program may not parse.

Even in languages where whitespace is not required for structure, it still matters for readability. Consistent indentation helps readers see block boundaries quickly.

3.2 Syntactic Rules

Syntax rules shape the skeleton of code. They define the forms of statements, such as assignment, if, while, for, return, and function definition. They also define expressions, such as arithmetic, comparisons, and function calls.

Syntax also includes precedence and associativity. Precedence decides which operator is applied first. For example, multiplication often has higher precedence than addition. Associativity decides how repeated operators group. For example, subtraction is often left associative, so $10 - 3 - 2$ is treated as $(10 - 3) - 2$.

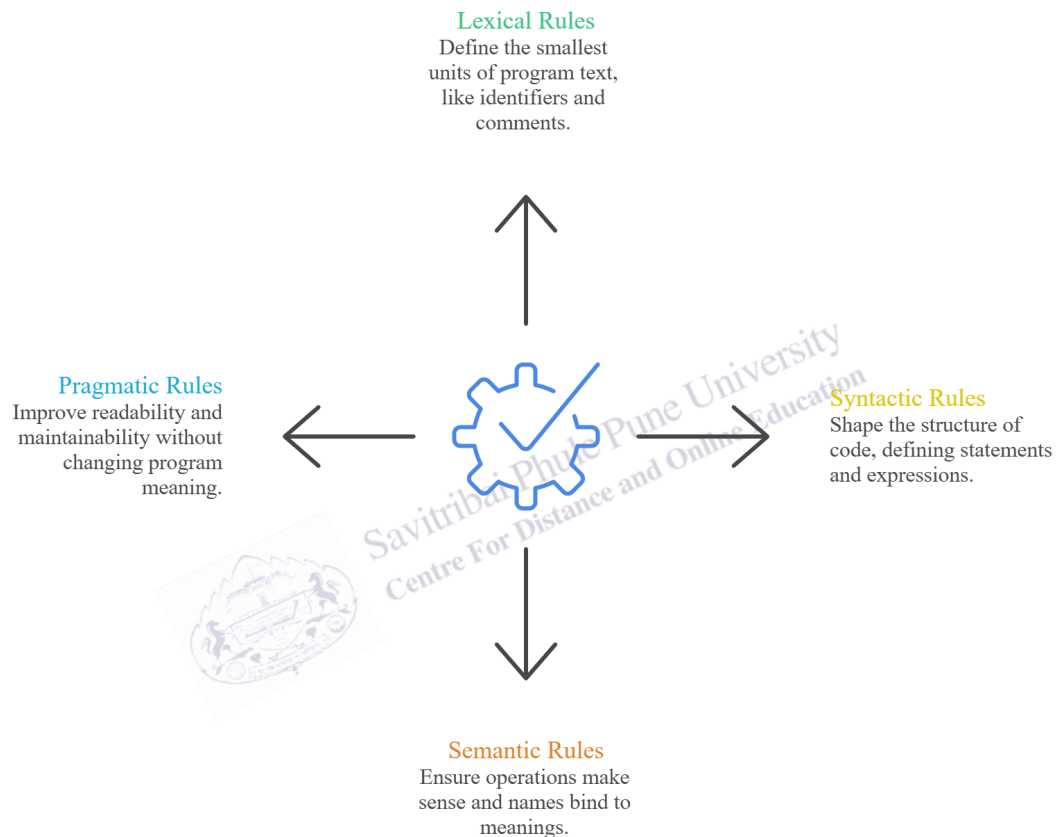
3.2.1 Statements and Expressions

An expression is a piece of code that evaluates to a value. A statement is a complete instruction that performs an action, such as updating a variable or controlling flow. Some languages blur this line, but the idea is still useful. If you understand whether a piece of text is an expression or a statement, you can predict where it can appear.

For example, in many languages, you can write an expression inside parentheses in an if condition. You cannot place a full statement there. So a rule may say: $\text{IfStmt} \rightarrow \text{if (Expr) Block}$. This rule makes the structure clear and avoids ambiguity.

Fig. 2

Types of Programming Rules



The image explains types of programming rules, including lexical, syntactic, semantic, and pragmatic rules. It shows how each rule type checks different aspects of code, from basic symbols to structure, meaning, and readability, helping programs remain correct and maintainable.

3.3 Semantic Rules

Semantic rules go beyond shape. They check whether operations make sense. They define how names bind to meanings, how values are stored, and how evaluation proceeds. Some semantic

rules are enforced at compile time, such as type checking in statically typed languages. Other semantic rules are enforced at run time, such as checking for division by zero.

3.3.1 Static Semantics

Static semantics includes rules that can be checked without executing the program. Type rules are a key part. For example, a rule may say that an integer can be added to an integer, producing an integer. A rule may also say that you cannot call a function with the wrong number of arguments.

Scope rules are also part of static semantics. They define where a name is visible. If you declare a variable inside a block, it may not be visible outside that block. A compiler can detect when a name is used without being declared.

3.3.2 Dynamic Semantics

Dynamic semantics describes what happens during execution. It includes the step by step behavior of expressions and statements. For example, it describes how a loop repeats, how a function call creates a new stack frame, and how exceptions change control flow. Dynamic semantics also includes run time checks in some languages, such as array bounds checking.

Understanding dynamic semantics helps students explain behavior that is not obvious from syntax alone. A program may be syntactically valid, but it can still crash because of a run time condition, such as dividing by zero or accessing a missing key in a map.

3.4 Pragmatic and Style Rules

Not all rules are enforced by the language engine. Many rules are pragmatic, meaning they are chosen to make work easier in practice. A style rule might say that function names use verbs, such as `computeTotal`. Another rule might say that constants are written in upper case, such as `MAX_SIZE`. These rules help readers learn the role of a name quickly.

Pragmatic rules also include testing and documentation rules. A team may require unit tests for each function and docstrings for public methods. These rules do not change the program's meaning, but they improve quality and reduce risk.

3.4.1 Readability and Maintainability

Readable program text supports long term success. A program often lives longer than expected. It may be used in new settings and edited by new people. Maintainability means the text can be

changed without breaking hidden assumptions. Rules support maintainability by limiting complexity. For example, a rule may limit function length or encourage small modules.



4.0 EXAMPLES AND CASE STUDIES

4.1 Example: A Simple Expression

Consider the expression $3 + 4$. The lexical rules say that “3” and “4” are number tokens and “+” is an operator token. The syntax rules say that `NUMBER + NUMBER` is a valid expression. The semantic rules say that addition on numbers produces a number value. When the expression is evaluated, it yields 7.

Now consider $3 + "4"$ in a language that does not allow mixing numbers and strings in addition. The syntax may still be valid, because it fits `Expr + Expr`. But the semantic rules may reject it as a type error. This example shows how syntax and semantics are different.

4.2 Example: An If Statement

Consider the following program text in simple form: `if x > 0 then print("positive")`. A language may require parentheses, or may require braces, or may require indentation. Those are syntax and lexical details. The meaning is that the program checks a condition and chooses an action. If the condition is true, it prints a message.

4.3 Example: A Function Definition

A function definition creates a reusable block of program text. A common structure is: `define f(n): return n * n`. Lexical rules define the tokens, including the keyword for defining a function, the identifier `f`, the parameter `n`, and the return keyword. Syntax rules define where colons, braces, or indentation appear.

Semantic rules define how parameters bind to arguments. If you call `f(5)`, the argument 5 binds to `n`, and the body evaluates to 25.

4.4 Case Study: Why a Program Fails to Compile

Consider a student who writes a short program in a C like language and forgets a semicolon at the end of a line. The compiler reports a syntax error, but the location may look confusing. The reason is that the parser tries to match the grammar and fails when it expects a token that is not there. The error message is a signal that the program text does not match the required structure.

In this case study, the fix is not to “guess” but to read the rule. The rule says an assignment statement ends with a semicolon. Adding the semicolon makes the text match the syntax. This

shows how rules serve as a guide. They are not just barriers. They are a map for constructing valid code.

4.5 Case Study: Linting and Formatting in a Team

In many teams, code is checked by tools before it is merged. A linter can warn about unused variables, confusing logic, or unsafe practices. A formatter can rewrite whitespace, line breaks, and indentation to match a style guide. These tools enforce pragmatic rules.

Suppose a team uses a rule that all files must pass a formatter. A student may feel that formatting is minor, but the team benefits. When every file follows the same style, code reviews can focus on logic, not spacing.

4.6 Case Study: Reading Program Text in a Security Review

Security work often begins with reading program text. Reviewers search for patterns such as unchecked input, unsafe string building, and missing access control checks. Many security bugs are not syntax bugs. They are semantic and pragmatic issues. The code may compile and run, but its meaning can allow an attacker to cause harm.

For example, a program may build a database query by joining strings that include user input. The syntax is valid, but the meaning can allow injection. A rule for safe programming might require parameterized queries. This is a pragmatic rule, enforced by policy and tools. Understanding rule types helps you explain why the code is risky and how to fix it.

In class projects, you should record the rule you used and the reason for it. When you cite a rule, you make your choices testable. This habit supports strong reports, clear code reviews, and better debugging, because others can follow your logic more easily.

1. Program text is a sequence of characters that must follow language rules to be a valid program.
2. Lexical rules form tokens, syntax rules create structure, and semantic rules define meaning and behavior.
3. Tools such as lexers, parsers, and compilers apply rules at different stages and give useful error messages.
4. Style guides and linters help keep code readable and easy to maintain, especially in team work and long projects.
5. Real case studies show that many failures come from broken rules, and many serious risks come from meaning errors rather than grammar mistakes.

5.0 Keyword Touch Vocabulary Meaning

Keyword	Touch Vocabulary Meaning	Use in This Topic
Program text	Written code characters	The source that tools read
Character	Single symbol like a letter	Builds all code text
Token	Grouped unit in code	Output of lexical analysis
Lexer	Token maker tool	Turns characters into tokens
Syntax	Structure rules	Defines valid statements
Grammar	Formal syntax description	Used by parsers
Parser	Structure checker	Builds a syntax tree
Semantics	Meaning rules	Explains what code does
Type	Category like int or string	Used in semantic checks
Scope	Where a name is visible	Controls name binding
Compiler	Translator tool	Produces target code
Interpreter	Direct executor	Runs code step by step
Lint	Style and risk checker	Finds patterns and warnings
Formatter	Automatic style tool	Makes code look consistent
Maintainability	Ease of change over time	Goal of good rules